UNIVERSITE NATIONALE DES SCIENCES TECHNOLOGIES INGÉNIERIE ET MATHÉMATIQUES (UNSTIM)

****

ÉCOLE NATIONALE SUPÉRIEURE DE GÉNIE MATHÉMATIQUE ET MODÉLISATION (ENSGMM)

****

ANALYSE ET BASES DE DONNÉES

****

Normalisation et Optimisation des bases de données : impact sur la cohérence et les performances

Réalisé par :

1. ADJIWANOU Kérol
2. GNONLONFOUN Octave
3. OUSMANE Aymane

Sous la supervision de :

Ing Aurel HANSINON

Novembre 2025

Table des matières

Introduction 2

I- Normalisation des bases de données : principes, enjeux et processus de normalisation 2

1- Objectifs de la normalisation 2

2- Processus de Normalisation 4

a- Analyse et conception initiale 4

b- Appliquer les formes normales 5

i) Première forme normale 1NF 5

ii) Deuxième forme normale 2NF 9

iii) Troisième forme normale 3NF 13

iv) Formes normales supérieures 16

c- Affiner la conception 16

3- Avantages et Inconvénients de la normalisation 16

II- Techniques d'assurance de la cohérence et d'optimisation des performances 20

A- Intégrité Référentielle 20

B- Indexation et accélération des requêtes 22

III- Cas pratique 24

Conclusion 24

Références bibliographiques 25

Introduction

Depuis plusieurs décennies, les bases de données sont devenues un pilier incontournable des systèmes informatiques modernes. Elles sont présentes dans presque tous les domaines : gestion, finance, santé, industrie, réseaux sociaux, commerce électronique, etc. Cette importance croissante est directement liée à l'explosion des volumes de données générés par les entreprises et les utilisateurs.

Pour gérer efficacement ces masses d'informations, des Systèmes de Gestion de Bases de Données (SGBD) ont été développés. Ils permettent de stocker, interroger, modifier et organiser l'information de manière fiable et sécurisée, tout en offrant aux utilisateurs l'illusion qu'ils sont les seuls à manipuler les données, malgré un partage parfois massif.

Une base de données ne se résume pas à un simple ensemble d'informations. C'est un modèle structuré, représentant une partie du monde réel, dans lequel les données doivent être interrogeables selon n'importe quel critère : retrouver tous les produits au-dessous d'un certain prix, tous les étudiants d'un département, ou encore les commandes d'un client.

Face à cette complexité, deux défis majeurs se posent aujourd'hui :

- **assurer la cohérence et la qualité des données**, grâce notamment à la normalisation ;
- **maintenir des performances élevées**, malgré des volumes grandissants et des requêtes de plus en plus complexes, d'où l'importance des mécanismes d'optimisation.

C'est dans cette perspective que s'inscrit notre étude : **comprendre en quoi la normalisation et les techniques d'optimisation contribuent à améliorer, à la fois, la qualité des données et les performances globales d'une base**.

I- Normalisation des bases de données : principes, enjeux et processus de normalisation

La normalisation est un processus consistant à organiser les données dans une base afin de réduire la **redondance**, améliorer la **cohérence**, et éviter les **anomalies** (insertion, suppression, mise à jour).

1- Objectifs de la normalisation

- ✓ Redondance des données
- ✓ Intégrité des données

- ✓ Anomalies des données.
- ✓ Performances et évolutivité
- ✓ Sécurité

La normalisation vise à organiser les données de manière rigoureuse afin de garantir la cohérence, la qualité et la maintenabilité d'une base de données. Elle répond à plusieurs objectifs essentiels.

- ✓ **Réduire la redondance des données**

La redondance, c'est-à-dire la répétition inutile d'informations dans plusieurs tables, entraîne une perte d'espace et surtout un risque élevé d'incohérence. Par exemple, si l'adresse d'un client est stockée à plusieurs endroits, une mise à jour partielle peut conduire à des informations contradictoires entre différentes interfaces ou rapports.

La normalisation élimine ce problème en s'assurant que chaque donnée est stockée **une seule fois**, dans une table clairement définie. Toutes les autres tables y font référence via des clés étrangères. Ainsi, une mise à jour modifie automatiquement l'information partout où elle est utilisée.

- ✓ **Améliorer l'intégrité des données**

Une base bien normalisée favorise l'application des règles d'intégrité, notamment l'intégrité référentielle.

Les clés étrangères empêchent la création d'enregistrements incohérents (ex. : une commande associée à un client inexistant) et empêchent aussi la suppression accidentelle d'informations encore utilisées.

L'intégrité est donc garantie **au niveau du SGBD lui-même**, ce qui réduit les bugs et renforce la fiabilité de l'application.

- ✓ **Éliminer les anomalies de manipulation**

Sans normalisation, on rencontre trois types d'anomalies :

- **Anomalie d'insertion** : impossibilité d'ajouter une donnée tant qu'une autre n'existe pas.
- **Anomalie de mise à jour** : la même information doit être modifiée à plusieurs endroits.
- **Anomalie de suppression** : supprimer une ligne entraîne la perte de données importantes.

En organisant les données pour que chaque fait soit représenté une seule fois, la normalisation supprime ces anomalies et simplifie les opérations de modification.

- ✓ **Améliorer les performances et l'évolutivité**

Contrairement à une idée répandue, la normalisation peut aussi améliorer les performances.

Des tables mieux structurées sont généralement plus petites, donc plus rapides à parcourir. Les index fonctionnent mieux sur des tables spécialisées que sur des tables très larges contenant des données redondantes.

De plus, une base normalisée est plus facile à **partitionner**, ce qui facilite son évolution lorsque le volume de données augmente.

- ✓ **Renforcer la sécurité**

La normalisation facilite la gestion des droits d'accès.

Les données sensibles sont stockées dans des tables spécifiques, bien identifiées, ce qui permet de contrôler précisément qui peut lire ou modifier ces informations.

L'audit et le suivi des modifications sont également simplifiés, car les données ne sont pas dispersées dans des duplications inutiles.

En résumé, la normalisation transforme un ensemble de données potentiellement redondantes et incohérentes en une base **fiable, logique et facile à maintenir**, capable de soutenir la croissance et la complexité d'une application.

2- Processus de Normalisation

La normalisation d'une base de données se déroule en plusieurs étapes, qui consistent à diviser une grande table en des tables plus petites et plus simples, tout en établissant des liens entre elles via des clés primaires et étrangères. Le processus se décompose en l'application des formes normales successives (1NF, 2NF, 3NF, etc.) pour éliminer la redondance des données et assurer l'intégrité des informations. Voici les étapes clés :

a- Analyse et conception initiale

- **Identifier le but et les exigences :** Déterminez l'objectif de la base de données et rassemblez toutes les informations nécessaires.
- **Diviser en tables initiales :** Regroupez les informations connexes pour créer des tables. Éliminez les groupes de répétition en créant une table distincte pour chaque jeu de données différent.

- **Identifier les colonnes et les clés primaires :** Définissez les colonnes pour chaque table et désignez une clé primaire unique pour chaque table. La clé primaire identifie sans ambiguïté chaque ligne.

b- Appliquer les formes normales

i) Première Forme Normale (1NF)

La Première Forme Normale impose qu'une table soit structurée de façon à ce que chaque colonne contienne des valeurs indivisibles et que chaque ligne soit clairement définie. C'est la base de toute normalisation.

➢ **Conditions pour atteindre la 1NF**

Une table est en 1NF lorsqu'elle respecte les critères suivants :

- **Unicité des lignes**

Chaque ligne doit être unique. Cela implique l'existence d'une clé primaire garantissant l'absence de doublons.

- **Absence de dépendance à l'ordre**

L'ordre des lignes et des colonnes n'a aucune importance. Le SGBD doit pouvoir retrouver les données indépendamment de leur position.

- **Atomicité des valeurs**

Chaque colonne doit contenir une **valeur atomique**, c'est-à-dire une seule information indivisible.
Cela interdit :

- les listes de valeurs
- les tableaux
- les ensembles
- les valeurs concaténées (ex. : "A,B,C")

- **Absence de colonnes répétées**

Aucune colonne ne doit être dupliquée.
Exemple à éviter : téléphone1, téléphone2, téléphone3.

Les deux premières conditions sont presque toujours respectées par les SGBD modernes. Les principales violations concernent donc l'atomicité et la présence de colonnes répétées.

- **Objectif central de la 1NF :** Garantir des données claires, propres et non ambiguës

- **Étapes pour transformer une table en 1NF**

Étape 1 : Identifier les groupes répétitifs

Repérer les colonnes qui contiennent :

- plusieurs valeurs séparées par des virgules
- des listes ou tableaux
- des colonnes multiples représentant le même attribut

Étape 2 : Décomposer en nouvelles lignes

Pour respecter l'atomicité, il faut créer autant de lignes que nécessaire afin que :

- chaque cellule contienne une seule valeur
- chaque colonne représente un unique attribut

Étape 3 : Vérifier la cohérence

Poser deux questions :

1. *Chaque colonne décrit-elle un seul attribut ?*
2. *Chaque ligne représente-t-elle un enregistrement unique ?*

Si oui, le tableau est en 1NF.

- **Exemples pratiques**

Exemple 1 : Commandes clients (colonnes répétées)

Table initiale :

CustomerID	Nom	Produit1	Produit2	OrderDate
1	Alice	Pommes	Bananas	2025-01-10
2	Bob	Oranges	Raisins	2025-01-11

Problème : colonnes Produit1 et Produit2 → même type de données.

Table en 1NF :

CustomerID	Nom	Produit	OrderDate
1	Alice	Pommes	2025-01-10
1	Alice	Bananas	2025-01-10
2	Bob	Oranges	2025-01-11
2	Bob	Raisins	2025-01-11

→ Les produits sont maintenant listés sous forme de lignes distinctes.

Exemple 2 : Notes d'étudiants (valeurs non atomiques)

Table initiale :

StudentID	Nom	Grades
101	Sarah	A, B, C
102	Michael	B, A, A

Problème : la colonne *Grades* contient plusieurs notes.

Table en 1NF :

StudentID	Nom	Grade
101	Sarah	A
101	Sarah	B
101	Sarah	C
102	Michael	B
102	Michael	A
102	Michael	A

→ Les notes sont maintenant atomiques et analysables individuellement.

Pourquoi la 1NF est importante ?

- Elle simplifie l'analyse et l'interrogation.
- Elle garantit des données claires, propres et non ambiguës.
- Elle prépare le terrain aux formes normales supérieures (2NF, 3NF…).

ii) Deuxième Forme Normale (2NF)

La Deuxième Forme Normale vise à éliminer les **dépendances partielles**, un problème typique des tables qui possèdent une **clé primaire composite**.

- **<u>Conditions pour atteindre la 2NF</u>**

Pour qu'une table soit en 2NF, elle doit d'abord être en 1NF :

- **Atomicité** : chaque cellule contient une seule valeur
- **Unicité des lignes** : clé primaire définie
- **Aucune colonne répétée**

- **<u>Objectif central de la 2NF</u> :** Eliminer les dépendances partielles

Une **dépendance partielle** survient lorsqu'un attribut non clé dépend **uniquement d'une partie d'une clé primaire composite**, au lieu de dépendre de la clé complète.

La 2NF impose donc que :

Tout attribut non clé doit dépendre de l'entièreté de la clé primaire.

Sinon, la table contient :

- redondances
- anomalies de mise à jour
- incohérences potentielles

- **Processus de décomposition en 2NF**

Pour vous assurer qu'un tableau respecte les règles de la 2NF, vous devez :

1. Identifiez toutes les clés candidates : Déterminez les ensembles minimaux d'attributs qui identifient de manière unique les tableaux. Ce sont vos clés de candidat.

2. Déterminer les dépendances fonctionnelles : Identifiez toutes les dépendances fonctionnelles dans le tableau. Plus précisément, recherchez les dépendances dans lesquelles les attributs non premiers (ceux qui ne font partie d'aucune clé candidate) dépendent uniquement d'une partie d'une clé composite.

3. Éliminez les dépendances partielles : Pour chaque dépendance partielle : Déplacez les attributs dépendants dans un nouveau tableau avec la partie de la clé dont ils dépendent. Assurez-vous que le nouveau tableau possède une clé primaire unique.

4. Répétez l'opération jusqu'à ce qu'il ne reste plus de dépendances partielles : Confirmez que chaque attribut non primaire de tous les tableaux dépend entièrement de sa clé primaire respective

- **Exemples pratiques**

Exemple 1 : Table d'inscription aux cours

Table initiale :

Identifiant de l'étudiant	ID du cours	Nom du cours	Nom de l'instructeur
1001	201	Principes fondamentaux de SQL	Ken Smith
1002	202	Introduction à Python	Merlin O'Donnell
1001	202	Introduction à Python	Merlin O'Donnell

Analyse

- **Clé primaire composite :** (StudentID, CourseID)
- **CourseName** dépend uniquement de CourseID
- **Instructor** dépend également uniquement de CourseID

Ce sont des **dépendances partielles** → la table **viole la 2NF**.

- **Décomposition en 2NF**

1. Table des inscriptions (relation principale)

Identifiant de l'étudiant	ID du cours
1001	201
1002	202
1001	202

Tableau des détails du cours

ID du cours	Nom du cours	Nom de l'instructeur
201	Principes fondamentaux de SQL	Ken Smith
202	Introduction à Python	Merlin O'Donnell

Maintenant :

- chaque attribut dépend totalement de la clé de sa table
- il n'y a plus de dépendance partielle → La structure est **en 2NF**

Exemple 2 : Table des commandes

Numéro d'ordre	ID du produit	Date de la commande	Nom du produit	Nom du fournisseur
1	201	2024-11-01	Ordinateur portable	TechSupply
1	202	2024-11-01	Souris	TechSupply
2	201	2024-11-02	Ordinateur portable	TechSupply
3	203	2024-11-03	Clavier	KeyMasters

Analyse

Clé composite : (OrderID, ProductID)

- **OrderDate** dépend de **OrderID seulement** → dépendance partielle
- **ProductName** dépend de **ProductID seulement** → dépendance partielle
- **Supplier** dépend aussi de **ProductID seulement** → dépendance partielle

→ La table **viole la 2NF**

- Décomposition en 2NF

1. Table Commande

Numéro d'ordre	Date de la commande
1	2024-11-01
2	2024-11-02
3	2024-11-03

Ensuite, nous créons un tableau qui contient des informations spécifiques à **ID du produit.**

Tableau des commandes

ID du produit	Nom du produit	Nom du fournisseur
201	Ordinateur portable	TechSupply
202	Souris	TechSupply
203	Clavier	KeyMasters

Table OrderItems (relation commande ↔ produit)

Numéro d'ordre	ID du produit
1	201
1	202
2	201
3	203

iii) Troisième Forme Normale (3NF)

La Troisième Forme Normale (3NF) est une étape essentielle de la normalisation. Elle vise à éliminer les **dépendances transitives**, c'est-à-dire les situations où un attribut non clé dépend d'un autre attribut non clé, au lieu de dépendre directement de la clé primaire.

Elle s'appuie sur les règles des formes normales précédentes :

- **1NF** : valeurs atomiques
- **2NF** : absence de dépendances partielles dans les clés composites

La 3NF va plus loin en garantissant que **tous les attributs non clés dépendent uniquement et directement de la clé primaire**.

C'est quoi une dépendance transitive ?

Il y a **dépendance transitive** lorsqu'un attribut A dépend de la clé primaire, mais passe par un autre attribut non clé B :

Clé primaire → B → A

Dans ce cas, A ne dépend pas directement de la clé primaire, ce qui introduit redondance, incohérences et anomalies.

- **<u>Condition pour qu'un tableau soit en 3NF</u>**

Une table est en 3NF si :

1. **Elle est en 2NF**
   (donc déjà atomique + sans dépendances partielles)
2. **Elle ne contient aucune dépendance transitive**
   → tout attribut non clé dépend *uniquement* de la clé primaire
   → aucun attribut non clé ne détermine un autre attribut non clé

- **<u>Objectif de la 3NF</u> :** Supprimer ces dépendances.

- **<u>Processus de décomposition en 3NF</u>**

  <u>Etape 1</u> : Identifier les dépendances transitives

Repérer les colonnes qui dépendent indirectement de la clé primaire.
En général, si un attribut peut être déterminé par une autre colonne non clé, c'est une dépendance transitive.

Étape 2 : Créer un nouveau tableau

Déplacer l'attribut ou le groupe d'attributs concernés dans une nouvelle table où ils dépendront directement d'une nouvelle clé primaire.

Étape 3 : Conserver la relation

Conserver dans la table d'origine la clé étrangère permettant de lier les deux tables.

- **Exemple pratique**

Tableau 3 : Inscriptions

ID d'Inscription	Nom de l'élève	ID du cours	Nom du cours
1001	Alice Smith	201	Principes fondamentaux de SQL
1002	Bob Green	202	Introduction à Python
1003	Charlie Blue	201	Principes fondamentaux de SQL

Analyse

- **Clé primaire** : ID d'inscription
- **ID du cours → Nom du cours**
- Mais la clé primaire est **ID d'inscription**, pas ID du cours.

Donc :

ID d'inscription → ID du cours → Nom du cours

Nom du cours dépend indirectement de la clé primaire
→ dépendance **transitive**
→ **violation de la 3NF**

Décomposition en 3NF

- Table Inscriptions (révisée)

ID d'Inscription	Nom de l'élève	ID du cours
1001	Alice Smith	201
1002	Bob Green	202
1003	Charlie Blue	201

- Table Cours

ID du cours	Nom du cours
201	Principes fondamentaux de SQL
202	Introduction à Python

Résultat

- **Chaque tableau contient uniquement des attributs dépendant directement de sa clé**
- Plus aucune dépendance transitive
- Structure propre, claire, évolutive

→ La base est désormais **en 3NF**

Pourquoi la 3NF est importante ?

- Évite la redondance inutile
- Empêche les incohérences lors des mises à jour
- Simplifie l'entretien de la base
- Rend le modèle plus logique et plus robuste
- Prépare la base à des opérations complexes (jointures, indexation…)

iv) Formes normales supérieures

Pour des besoins plus complexes, vous pouvez appliquer des formes normales plus avancées, telles que la Forme Normale de Boyce-Codd (BCNF), la 4NF (basée sur les dépendances multivaluées) et la 5NF (basée sur les dépendances de jointure).

c- Affiner la conception

- **Établir les relations :** Configurez les relations entre les tables à l'aide de clés étrangères (des colonnes qui se réfèrent aux clés primaires d'autres tables).
- **Vérifier et optimiser :** Relisez et affinez votre conception pour vous assurer qu'elle est cohérente, efficace et qu'elle répond aux exigences initiales tout en éliminant les redondances et les anomalies de mise à jour et de suppression.

3- Avantages et Inconvénients de la normalisation

La normalisation apporte des bénéfices considérables en termes de cohérence et de qualité des données, mais elle introduit également des défis, notamment en matière de performance et de complexité. Elle n'est donc pas une solution absolue : il faut trouver un équilibre entre rigueur théorique et besoins pratiques.

❖ **<u>Avantages de la normalisation</u>**

1. Réduction de la redondance

La normalisation garantit que chaque information est stockée une seule fois.
Cela réduit :

- le gaspillage de stockage,
- les duplications inutiles,
- les incohérences entre tables.

Exemple :
Mettre à jour l'adresse d'un client se fait en un seul endroit, et non dans plusieurs tables différentes.

2. Amélioration de la cohérence des données

Avec une seule source fiable pour chaque information, les contradictions deviennent impossibles.
Les anomalies de mise à jour, de suppression ou d'insertion sont éliminées.

Toute l'application repose sur une base cohérente, ce qui renforce la fiabilité des traitements et des analyses.

3. **Mises à jour plus simples et plus fiables**

La normalisation réduit le risque d'erreurs lors des opérations CRUD.
Une seule ligne modifiée suffit pour répercuter un changement dans toute la base grâce aux clés étrangères.

→ Moins de bugs
→ Moins de logique "patchée" dans l'application

4. **Sécurité renforcée**

Les données sensibles peuvent être isolées dans des tables spécifiques.
Cela permet :

- d'appliquer des règles d'accès ciblées,
- d'éviter la dispersion d'informations critiques (ex : cartes bancaires),
- de simplifier l'audit de sécurité.

5. **Meilleure évolutivité**

Des tables plus petites et mieux organisées :

- accélèrent les index,
- facilitent la partition,
- optimisent les caches.

La base peut s'adapter plus facilement à une croissance importante du volume de données.

6. **Collaboration plus efficace**

Un schéma clair et cohérent permet :

- aux développeurs de comprendre plus rapidement la structure,
- aux analystes d'écrire des requêtes fiables,
- aux administrateurs de faire des optimisations sans risque.

7. **Sauvegarde et restauration simplifiées**

Avec des données bien structurées et non dispersées :

- les backups sont plus fiables,
- la restauration garantit l'intégrité référentielle,
- le risque de données "orphelines" est très réduit.

❖ Inconvénients et défis de la normalisation

1. Complexité accrue des requêtes

Pour obtenir une information complète, il faut souvent effectuer plusieurs `JOIN`.

Exemple :
Afficher l'historique des commandes d'un client nécessite parfois 3 à 5 jointures.

Conséquences :

- requêtes plus difficiles à écrire,
- risque d'erreur plus élevé,
- temps d'exécution potentiellement plus long.

2. Performance parfois réduites

Les jointures répétées imposent une charge supplémentaire :

- recherches dans plusieurs tables,
- accès disque multiples,
- surcharge de tri et de regroupement.

Dans les systèmes à forte intensité de lecture (analytics, dashboards), cela peut ralentir l'affichage.

3. Temps de développement plus élevé

Une base normalisée nécessite :

- de comprendre les relations entre toutes les tables,
- de maîtriser les clés étrangères,
- de savoir composer des requêtes complexes.

Ce qui pouvait être un simple `SELECT` devient un `JOIN` à 4 tables.

4. Risque de surnormalisation

Aller trop loin dans la normalisation peut créer une **complexité artificielle**.

Exemple :
Séparer prénom, second prénom et nom dans trois tables différentes.
→ inutile, compliqué, peu pratique.

Un cas réel :
Un site e-commerce avait normalisé les catégories en 6 niveaux →
Résultat : une requête simple (“tous les appareils électroniques”) nécessitait 7 jointures et prenait plusieurs secondes.
La théorie parfaite devient alors un handicap.

5. **Mauvais choix pour les applications à forte lecture**

Dans certains systèmes (réseaux sociaux, flux temps réel, reporting),
la normalisation optimise l’écriture au détriment de la lecture, alors que 95 % des opérations sont des lectures.

Dans ce cas, une **dénormalisation stratégique** est préférable.

6. **Surcharge de maintenance**

Plus il y a de tables, plus il faut gérer :

- d’index,
- de clés étrangères,
- de règles de dépendance,
- de backups interconnectés.

Cela augmente la charge administrative de la base.

Résumé

La normalisation doit trouver **un juste équilibre** :

- assez normalisée pour assurer cohérence et intégrité,
- pas trop pour ne pas sacrifier les performances et la simplicité.

La bonne approche consiste à normaliser **jusqu’à la 3NF**, puis dénormaliser légèrement **selon les besoins réels de performance**.

II- Techniques d'assurance de la cohérence et d'optimisation des performances

A- Intégrité Référentielle

L'intégrité référentielle est un mécanisme fondamental dans les bases de données relationnelles. Elle garantit la cohérence des relations entre tables en contrôlant la validité des liens établis via les clés étrangères. Contrairement à la normalisation, qui organise les données au niveau conceptuel, l'intégrité référentielle agit comme une règle technique appliquée par le SGBD pour empêcher les incohérences et protéger la qualité des données.

1. **Définition de l'intégrité référentielle**

L'intégrité référentielle impose qu'une valeur de clé étrangère référence toujours une clé primaire existante dans une autre table. Cela signifie :

- un enregistrement enfant ne peut pas exister sans son parent ;
- il est impossible d'insérer un enfant qui pointe vers une clé inexistante ;
- certaines suppressions ou modifications sont bloquées si elles mettent en danger la cohérence des données.

Le SGBD applique ces règles automatiquement, indépendamment du code de l'application.

2.Les contraintes référentielles (actions du SGBD)

Lorsqu'une modification concerne une clé primaire, le SGBD peut réagir de différentes manières selon la règle définie :

ON DELETE / ON UPDATE RESTRICT

- ✓ Empêche l'action si elle entraîne une incohérence.
- ✓ Impossible de supprimer un client qui a des commandes.
- ✓ Bon pour préserver l'historique.

ON DELETE / ON UPDATE CASCADE

- ✓ Propage automatiquement la suppression ou la modification aux tables enfants.
- ✓ Supprimer un client supprime aussi toutes ses commandes.
- ✓ Pratique pour les données fortement dépendantes.

ON DELETE SET NULL

- ✓ La suppression du parent met la FK (Foreign Key) à NULL dans la table enfant.
- ✓ Utile lorsque la relation n'est pas obligatoire.

ON DELETE SET DEFAULT

- ✓ Affecte une valeur par défaut lorsqu'un parent est supprimé.
- ✓ Rare, mais utile dans certains contextes métiers.

3. Prévention des erreurs et des données orphelines

L'intégrité référentielle empêche :

- ✓ l'insertion d'une commande associée à un client inexistant ;
- ✓ la suppression d'un client qui possède encore des commandes (sauf CASCADE) ;
- ✓ la mise à jour incorrecte de clés primaires ;
- ✓ la création de valeurs incohérentes dans le schéma.

Elle protège donc la base contre les anomalies fonctionnelles, même dans les systèmes où plusieurs applications accèdent aux données simultanément.

4. Impact sur la cohérence et la qualité des données

L'intégrité référentielle garantit que :

- ✓ les relations entre entités restent valides ;
- ✓ les données sont fiables et structurées ;
- ✓ les analyses, rapports et tableaux de bord ne contiennent pas de valeurs incohérentes ;
- ✓ le SGBD s'assure lui-même du respect des règles métiers.

Elle joue un rôle essentiel dans la confiance que l'on peut accorder à l'information stockée.

5. Impact indirect sur les performances

Bien que son objectif principal soit la cohérence, l'intégrité référentielle peut aussi améliorer indirectement les performances :

- ✓ Les données propres et bien reliées réduisent les corrections et incohérences coûteuses.
- ✓ Les requêtes JOIN deviennent plus fiables et optimisées.
- ✓ Les index sur PK et FK facilitent l'accès aux lignes liées.

Dans les systèmes complexes, la garantie d'intégrité évite les traitements supplémentaires pour vérifier la cohérence manuellement dans le code.

B- Indexation et accélération des requêtes

L'indexation est une technique d'optimisation essentielle dans les bases de données relationnelles. Elle permet d'améliorer de manière significative la vitesse d'exécution des requêtes, en particulier lorsqu'il s'agit de rechercher, filtrer, trier ou joindre de grandes quantités de données. Contrairement à la normalisation et à l'intégrité référentielle, qui se concentrent sur l'organisation logique et la cohérence, l'indexation agit directement sur **les performances**.

1. Définition de l'indexation

Un index est une structure de données auxiliaire (généralement un arbre B-tree ou un hash) créée sur une ou plusieurs colonnes d'une table. Il fonctionne comme l'index d'un livre : au lieu de parcourir toutes les pages (toutes les lignes de la table), le SGBD utilise l'index pour accéder directement aux données recherchées.

Sans index, une requête effectue un **full table scan**, c'est-à-dire qu'elle lit toute la table ligne par ligne, ce qui devient très lent lorsque la table contient des millions de lignes.

2. Comment fonctionne un index ?

La plupart des SGBD utilisent des **arbres B-Tree** :

- les données sont triées dans l'index ;
- les recherches deviennent logarithmiques (O(log n)) au lieu de linéaires (O(n)) ;
- les performances des `WHERE`, `JOIN`, `ORDER BY`, `GROUP BY` sont considérablement améliorées.

D'autres types d'index existent selon les besoins (hash index, full-text, index géospatiaux, etc.), mais le principe reste le même : **accéder plus vite aux lignes pertinentes**.

3. Quand utiliser un index ?

L'indexation accélère les requêtes dans les cas suivants :

- colonnes souvent utilisées dans les clauses `WHERE` (filtrage) ;
- colonnes impliquées dans les `JOIN` ;
- colonnes utilisées dans `ORDER BY` ou `GROUP BY` ;
- recherches de texte sur de grands volumes ;
- colonnes de référence (clé étrangère).

Ajouter un index sur une colonne rarement utilisée dans les requêtes n'a pas d'intérêt.

4. Les types d'index

- **Index simple**

Index créé sur une seule colonne.
Ex. : `CREATE INDEX idx_nom ON clients(nom);`

- **Index composite**

Index basé sur plusieurs colonnes, utile pour optimiser des requêtes multi-conditions.
Ex. : `CREATE INDEX idx_client_date ON commandes(id_client, date_commande);`

- **Index unique**

Empêche la duplication de valeurs.
Utilisé pour les emails, numéros de téléphone, identifiants uniques…

- **Index sur clés étrangères**

Très recommandés, car ils accélèrent les jointures fréquentes entre tables liées.

- **Index full-text**

Optimise les recherches de texte ("contient", "commence par", "mot clé").

5. Bonnes pratiques pour les méthodes d'indexation

- Toujours indexer les clés étrangères.
- Indexer les colonnes qui filtrent beaucoup de données.
- Ne pas indexer les colonnes avec faible sélectivité (ex : genre, statut oui/non).
- Utiliser des index composites lorsque les requêtes portent souvent sur plusieurs colonnes.
- Vérifier régulièrement l'utilisation réelle des index (analyse d'exécution, plan de requête).
- Supprimer les index inutiles ou redondants.

6. Impacts de l'indexation sur les performances

L'indexation bien pensée peut réduire le temps de réponse :

- d'une requête complexe passant de **15 secondes à quelques millisecondes** ;
- d'une jointure sur grandes tables passant de **plusieurs millions** de lignes scannées à quelques milliers.

Elle constitue donc l'un des leviers les plus efficaces pour optimiser les requêtes sans modifier le schéma logique de la base.

L’indexation est un mécanisme essentiel pour accélérer les requêtes dans les bases de données relationnelles. Elle permet de garantir des performances élevées, en particulier lorsque le volume de données augmente. Cependant, son efficacité dépend d’une utilisation judicieuse et adaptée aux besoins du système : trop d’index ralentissent les écritures, trop peu d’index ralentissent les lectures.

III- Cas pratique

(Voir Notebook Jupyter)

Conclusion

La normalisation et l’optimisation représentent deux piliers essentiels dans la conception et la gestion des bases de données modernes. La normalisation, à travers ses différentes formes, permet de structurer efficacement l’information, d’éliminer les redondances et de prévenir les anomalies susceptibles de compromettre la cohérence des données. Toutefois, cette rigueur doit être accompagnée de techniques d’optimisation capables de garantir des performances adaptées aux besoins croissants des systèmes, notamment grâce à l’indexation, au contrôle de l’intégrité référentielle ou à la réécriture intelligente des requêtes. Ainsi, la qualité d’une base de données ne repose pas uniquement sur son degré de normalisation, ni uniquement sur sa rapidité, mais sur la capacité à trouver un équilibre pertinent entre cohérence et performance. Le cas pratique présenté montre d’ailleurs que cette complémentarité est indispensable dans les contextes réels où les volumes de données augmentent et les attentes des utilisateurs s’intensifient. Maîtriser ces deux dimensions, c’est donc garantir la fiabilité, l’efficacité et la pérennité de tout système d’information.

Références bibliographiques

- https://www.datacamp.com/fr/tutorial/normalization-in-dbms
- https://www.ibm.com/fr-fr/think/topics/database-normalization
- https://www.purestorage.com/fr/knowledge/what-is-data-normalization.html#:~:text=La%20normalisation%20assure%20une%20meilleure,des%20jointures%20sur%20les%20tables.
- https://hal.science/hal05033888v1/file/MAGIC%20of%20Data%20Management%202024%20FR.pdf
- Gardarin-Livre-TotalBD_090107.pdf
- https://www.datacamp.com/fr/tutorial/first-normal-form
- https://www.datacamp.com/fr/tutorial/first-normal-form
- https://www.datacamp.com/fr/tutorial/second-normal-form